

DOCUMENTAZIONE DEL PROGETTO DI RETI INFORMATICHE

623720 Lorenzo Salvatelli

Questo progetto realizza un gioco distribuito Trivia Quiz con architettura client-server. L'obiettivo è fornire un servizio in cui più client, connessi via TCP, scelgono un tema, rispondono a una sequenza di domande e ricevono punteggi aggiornati in tempo reale. La scelta progettuale principale è stata basata su semplicità d'implementazione, chiarezza del protocollo e reattività per l'utente.

File di progetto

Il progetto si sviluppa su più file: header.h per definizioni di macro e strutture dati. server.c dove si snoda il flusso di esecuzione dello stesso, client.c con il rispettivo flusso. s_utilityfunctions.c che contiene le implementazioni delle funzioni di utilità lato server, c_utilityfunctions.c per il client, both_utilityfunctions.c per entrambi.

Scelta dell'architettura e motivazioni

Ho adottato un server concorrente multithread. Al verificarsi di una nuova connessione il processo server crea un thread dedicato che gestisce l'intera sessione del client: registrazione, scelta del quiz, invio delle domande, ricezione delle risposte e aggiornamento del punteggio. Per il contesto didattico e per il carico previsto (pochi client simultanei), tale modello offre un ottimo compromesso tra semplicità e reattività: ogni client procede senza essere bloccato dalle operazioni degli altri.

L'applicazione usa TCP per la comunicazione: la scelta di un canale orientato alla connessione è motivata dalla necessità di consegna ordinata e affidabile dei messaggi testuali (domande, risposte e comandi). Sopra TCP è definito un semplice protocollo di comunicazione che consente di determinare esattamente quanti byte leggere per ogni messaggio e riduce i problemi legati a ricezioni parziali.

Protocollo di comunicazione

Il protocollo implementato è volutamente minimale e strutturato in quattro fasi:

1. invio di un campo tipo a lunghezza fissa (CMD_LEN)
2. invio della lunghezza del payload
3. invio del payload testuale
4. scambio di un ACK a lunghezza fissa (ACK_LEN).

Il campo tipo rende immediata l'interpretazione del contenuto (es. QST per domanda, NSW per risposta), mentre la lunghezza del payload elimina ambiguità sulle dimensioni del messaggio.

L'ACK finale sincronizza mittente e ricevente e segnala che il messaggio è stato ricevuto correttamente prima di procedere oltre.

Flusso operativo

All'avvio il server crea il socket listener e un thread `input_routine` che campiona la console per comandi dell'operatore («exit»). Ciclicamente il main chiama `accept` e, per ogni nuovo client, alloca memoria per il socket descriptor e crea un thread `thread_routine` che prende possesso della sessione. La routine per client gestisce i passaggi di registrazione, la scelta del tema, il caricamento delle domande dal file corrispondente e l'invio sequenziale delle domande con valutazione delle risposte ricevute. Il client mostra un menu iniziale: avvia quiz o esce. Se sceglie di partecipare si connette al server, invia il nickname, riceve le domande, può invocare comandi speciali («show score», «endquiz») e risponde alle domande. In caso di SIGINT il client invia un messaggio di disconnessione e chiude ordinatamente il socket.

Analisi critica e miglioramenti possibili

Il progetto che ho implementato soddisfa i requisiti richiesti: protocollo per la lettura di messaggi, caricamento domande da file, gestione di comandi speciali e notifica di disconnessione client->server. Mentre la notifica di disconnessione server->client si basa su un errore di comunicazione e non su un messaggio vero e proprio (per fare ciò sarei dovuto ricorrere ad un client con multiplexing).

Il modello concorrente, inoltre, rende il codice semplice da comprendere e verificare in ambiente didattico.

Tuttavia questo approccio ha dei limiti: con centinaia o migliaia di client il modello diventa inefficiente per memoria e scheduling. Un server basato su I/O multiplexing sarebbe più scalabile. L'uso di `pthread_cancel` può lasciare risorse non liberate, sarebbe meglio passare a terminazione cooperativa con variabili atomiche o condition variable, per cleanup di socket e mutex. Si usa inoltre un solo mutex per tutte le strutture condivise, mentre sarebbe adeguato, all'aumentare dell'utilizzazione, di istanziarne diversi per diverse semantiche. Infine, alcune pratiche adottate (uso di `system("clear")`, `sleep` e `goto`) sono utili e funzionali, ma da rivedere per applicazioni di più larghe vedute.